

หน้า 1 / 65

โมดูล 5: การประมวลผล (คอนเทนเนอร์และ Serverless) – Computing (Containers and Serverless)

หน้า 2 / 65

วัตถุประสงค์ของโมดูล

เมื่อจบโมดูลนี้ คุณจะสามารถ:

- อธิบายวัตถุประสงค์และการทำงานของ AWS Lambda
- อธิบายวัตถุประสงค์และการทำงานของ application programming interfaces (APIs) รวมถึง RESTful APIs
- อธิบายประโยชน์และการทำงานของ Amazon API Gateway
- อธิบายวัตถุประสงค์และการทำงานของคอนเทนเนอร์ (containers) และบริการ AWS ที่รองรับการใช้คอนเทนเนอร์
- อธิบายวัตถุประสงค์และการทำงานของ AWS Step Functions

หน้า 3 / 65

ภาพรวมของโมดูล

หัวข้อ:

- AWS Lambda
- APIs และ REST
- Amazon API Gateway
- คอนเทนเนอร์บน AWS (Containers on AWS)
- AWS Step Functions
- ตรวจสอบความรู้ (Knowledge check)

การสาธิตโดยผู้สอน:

- deploy Docker Containers บน Amazon Elastic Container Service (Amazon ECS)

แล็บ: ไม่มี (None)

แบบฝึกหัด / โปรเจกต์:

- กิจกรรมที่ 5: สร้าง Lambda Function เพื่อส่งรายงานประจำวันทางอีเมล
- โปรเจกต์: Troubleshooting Playbook

หน้า 4 / 65

ฐานความรู้สำหรับการแก้ไขปัญหา – ประเด็นของโมดูล 5

- เปิดสำเนาแม่แบบฐานความรู้สำหรับการแก้ไขปัญหาของคุณไว้ในโหมดแก้ไข
- สร้างรายการใหม่ทุกครั้งที่คุณเจอสถานการณ์ที่ต้องแก้ไขปัญหา
- หมวดหมู่ที่เน้นในโมดูลนี้:
 - Compute (การประมวลผล)

หน้า 5 / 65

โมดูล 5: การประมวลผล (คอนเทนเนอร์และ Serverless)

AWS Lambda

หน้า 6 / 65

การประมวลผลแบบ Serverless (Serverless computing)

การ deploy และปฏิบัติการแบบดั้งเดิม (Traditional)	การ deploy และปฏิบัติการแบบ Serverless
จัดเตรียม (provision) instance	ไม่ต้องทำ (AWS ดูแล)
อัปเดตระบบปฏิบัติการ (OS)	ไม่ต้องทำ (AWS ดูแล)
ติดตั้งแพลตฟอร์มแอปพลิเคชัน	ไม่ต้องทำ (AWS ดูแล)
สร้างและ deploy แอปพลิเคชัน	สร้างและ deploy แอปพลิเคชัน
ตั้งค่าการขยายขนาดอัตโนมัติและการกระจายโหลด	ไม่ต้องทำ (AWS ดูแล)
ติดตั้งแพตช์รักษาความปลอดภัย และเฝ้าติดตามเซิร์ฟเวอร์เป็นประจำ	ไม่ต้องทำ (AWS ดูแล – ในสโลคคือ "Continuously patch, secure, and monitor servers" ที่ทำเป็นสัจจา)
เฝ้าติดตามและดูแลแอปพลิเคชัน	เฝ้าติดตามและดูแลแอปพลิเคชัน

ในสโลค ผัง Serverless แสดงรายการที่ผู้ใช้ไม่ต้องทำเป็นตัวอักษรสีจาง เหลือเพียง "Build and deploy applications" และ "Monitor and maintain applications" ที่ผู้ใช้ยังต้องทำ

หน้า 7 / 65

AWS Lambda

AWS Lambda ช่วยให้ทำงานได้โดยไม่ต้องจัดเตรียมหรือจัดการเซิร์ฟเวอร์

- 1. อัปโหลดโค้ดของคุณขึ้น AWS Lambda (เป็น Lambda function)
- 2. ตั้งค่าให้โค้ดถูกทริกเกอร์จากบริการ AWS อื่น ๆ (AWS SERVICES), HTTP endpoints (HTTP ENDPOINTS) หรือกิจกรรมในแอปพลิเคชัน (MOBILE APPS)
- 3. AWS Lambda รันโค้ดของคุณเฉพาะเมื่อถูกทริกเกอร์เท่านั้น
- 4. จ่ายตามเวลาประมวลผล (compute time) ที่ใช้

หน้า 8 / 65

คุณสมบัติหลักของ AWS Lambda

คุณสมบัติหลักของ AWS Lambda ได้แก่:

- นำโค้ดของคุณเองมาใช้ (Bring your own code)
- การดูแลระบบเป็นอัตโนมัติทั้งหมด (Completely automated administration)
- ทนทานต่อความผิดพลาดในตัว (Built-in fault tolerance)
- ขยายขนาดอัตโนมัติ (Automatic scaling)
- ประสานการทำงานของหลาย function (Orchestrate multiple functions)
- โมเดลความปลอดภัยแบบผสานรวม (Integrated security model)
- จ่ายตามการใช้งาน (Pay per use)
- โมเดลทรัพยากรที่ยืดหยุ่น (Flexible resource model)

หน้า 9 / 65

ขั้นตอนการ deploy AWS Lambda

- #1 สร้างคลาส Lambda handler ในโค้ดของคุณ
- #2 สร้าง AWS Lambda function
- #3 อนุญาตให้ AWS Lambda รับบทบาท (assume) IAM role
- #4 อัปโหลดโค้ด
- #5 เรียกใช้ (invoke) AWS Lambda function
- #6 เฝ้าติดตาม function ของคุณ (ด้วย Amazon CloudWatch)

หน้า 10 / 65

ตัวอย่าง AWS Lambda: เริ่มและหยุด Amazon EC2 instances หยุด instances (Stop instances):

- ตั้งตารางเวลาให้ทริกเกอร์ AWS Lambda หยุด instances ตอนกลางคืน
- AWS Lambda function (ที่มี Role) ถูกทริกเกอร์ให้หยุด EC2 instances
- Amazon EC2 instances เข้าสู่สถานะ stopped

เริ่ม instances (Start instances):

- ตั้งตารางเวลาให้ทริกเกอร์ AWS Lambda เริ่ม instances ตอนเช้า
- AWS Lambda function (ที่มี Role) ถูกทริกเกอร์ให้เริ่ม EC2 instances
- Amazon EC2 instances เข้าสู่สถานะ running

หน้า 11 / 65

Lambda layers

Lambda layers ช่วยให้นักพัฒนา:

- ตั้งค่า Lambda function ให้ใช้ไลบรารีที่ไม่ได้รวมอยู่ใน deployment package
 - Lambda layer คือไฟล์ ZIP ที่มีไลบรารีและ custom runtime
- ทำให้ deployment package มีขนาดเล็ก
- หลีกเลี่ยงข้อผิดพลาดด้าน dependency ของแพ็คเกจในโค้ด
- แชร์ไลบรารีกับนักพัฒนาด้านอื่น

หน้า 12 / 65

ขีดจำกัดของ AWS Lambda (AWS Lambda limits)

ขีดจำกัดของ AWS Lambda ได้แก่:

- ขีดจำกัดทรัพยากรต่อการเรียกใช้แต่ละครั้ง เช่น หน่วยความจำ runtime ระยะเวลา
- ขีดจำกัดของบัญชีต่อ Region เช่น การทำงานพร้อมกัน (concurrency)
- ขีดจำกัดการ deploy เช่น ขนาดแพ็คเกจ

function ที่เกินขีดจำกัดใดก็ตามจะล้มเหลวพร้อม exception ว่าเกินขีดจำกัด (exceeded limits exception)

หน้า 13 / 65

ประเด็นสำคัญ (Key Takeaway)

- การประมวลผลแบบ serverless ช่วยให้คุณสร้างและรันแอปพลิเคชันและบริการได้โดยไม่ต้องจัดเตรียมหรือจัดการเซิร์ฟเวอร์
- AWS Lambda มีความทนทานต่อความผิดพลาดและการขยายขนาดอัตโนมัติในตัว
- หน่วยความจำสูงสุดที่จัดสรรให้ Lambda function หนึ่งตัวคือ 3 GB
- เวลาทำงานสูงสุดของ Lambda function คือ 15 นาที

หน้า 14 / 65

โมดูล 5: การประมวลผล (คอนเทนเนอร์และ Serverless)

APIs และ REST

หน้า 15 / 65

APIs

application programming interface (API):

- ให้การเข้าถึงแอปพลิเคชันผ่านโปรแกรม ตัวอย่างเช่น มักใช้กับโปรแกรมที่สื่อสารกันเอง
- รองรับหลายภาษาโปรแกรม เช่น API สำหรับ Java, API สำหรับ C++ เป็นต้น

แผนภาพ: Client → คำขอ (Request) → API interface → Application → ผลตอบกลับ (Response) → Client

หน้า 16 / 65

RESTful APIs

Representational state transfer (REST):

- การสื่อสารผ่าน HTTP
- เปิดเผยแพร่ทรัพยากรที่ URL เฉพาะ
- web APIs จำนวนมากเป็นแบบ RESTful
- เป็นการออกแบบสำหรับแอปพลิเคชันบนเครือข่ายที่เชื่อมโยงกันแบบหลวม (loosely coupled)

หลักการออกแบบ RESTful 6 ข้อ:

- อินเทอร์เฟซที่เป็นรูปแบบเดียวกัน (A uniform interface)
- ไม่เก็บสถานะ (Stateless)
- สถาปัตยกรรมแบบไคลเอนต์-เซิร์ฟเวอร์ (Client-server architecture)
- แคชได้ (Cacheable)
- ระบบแบบชั้น (Layered system)
- โค้ดตามต้องการ (Code on demand – ไม่บังคับ)

หน้า 17 / 65

คำขอ REST (REST requests)

ส่วนประกอบของคำขอ REST ได้แก่:

- Endpoint (ในรูปของ URL)
- Method:
 - GET – เพื่ออ่านทรัพยากร
 - POST – เพื่อสร้างทรัพยากร
 - PUT – เพื่ออัปเดตทรัพยากรที่มีอยู่
 - DELETE – เพื่อลบทรัพยากร
- Header:
 - ตัวอย่าง: -H "Content-Type: application/json"
- Data (body):
 - โดยทั่วไปเป็นส่วนหนึ่งของคำขอ POST และ PUT

(แผนภาพ: URL, Method, Headers, Body)

หน้า 18 / 65

ตัวอย่าง REST ด้วย cURL

คำขอ (Request):

```
curl -i \
-X POST \
-d @file.json \
-H "Content-Type: application/json" \
https://www.example.com/someresource/
```

ผลตอบกลับ (Response):

```
{
  "Status": "Accepted"
}
```

(แผนภาพ: Client → HTTPS → API interface → Application)

หน้า 19 / 65

รหัสสถานะ HTTP (HTTP status codes)

- 100s – การตอบกลับเชิงข้อมูล (Informational response)
- 200s – สำเร็จ (Success)
- 300s – การเปลี่ยนเส้นทาง (Redirection)
- 400s – ข้อผิดพลาดฝั่งไคลเอนต์ (Client errors)
- 500s – ข้อผิดพลาดฝั่งเซิร์ฟเวอร์ (Server errors)

หน้า 20 / 65

โมดูล 5: การประมวลผล (คอนเทนเนอร์และ Serverless)

Amazon API Gateway

หน้า 21 / 65

Amazon API Gateway

Amazon API Gateway:

- ช่วยให้นักพัฒนาสร้างและดูแล APIs ตัวอย่างเช่น สร้าง REST API ให้แอปพลิเคชันที่คุณรันบน AWS
- เป็นบริการแบบมีผู้ดูแลให้เต็มรูปแบบ (fully managed) ซึ่งจัดการ:
 - การขยายขนาด (Scaling)
 - การควบคุมการเข้าถึง (Access control)
 - การเฝ้าติดตาม (Monitoring)

หน้า 22 / 65

การ deploy Amazon API Gateway

แผนภาพ: Applications หลายตัว ↔ API (Frontend) ของ Amazon API Gateway → Backend ได้แก่ AWS Lambda, Amazon DynamoDB, Amazon EC2, Amazon VPC และ Internet

บริการ AWS ประมวลผลคำขอและผลตอบกลับของการผสานรวม (AWS services processing integration requests and responses)

หน้า 23 / 65

ประโยชน์ของ API Gateway

ประโยชน์ของ API Gateway ได้แก่:

- ได้ประโยชน์จากต้นทุนต่ำและประสิทธิภาพ
- ให้สมรรถนะได้ทุกระดับขนาด

- เฝ้าติดตามกิจกรรมของ API ได้ง่าย
- ทำให้การพัฒนา API คล่องตัวขึ้น
- ใช้การควบคุมความปลอดภัยที่ยืดหยุ่น
- รัน APIs โดยไม่ต้องมีเซิร์ฟเวอร์
- สร้าง RESTful API endpoints ให้บริการที่มีอยู่

หน้า 24 / 65

ตัวอย่าง API Gateway (1 จาก 2)

แผนภาพ Serverless Web Application (Client เชื่อมต่อกับส่วนต่าง ๆ):

- **Amazon Simple Storage Service (S3)** – พื้นที่จัดเก็บแบบออบเจกต์และการโฮสต์เว็บไซต์: Amazon S3 โฮสต์เนื้อหาเว็บไซต์แบบสแตติก เช่น HTML, CSS, JavaScript
- **Amazon Cognito** – ไดรกทอรีผู้ใช้และการยืนยันตัวตน: Amazon Cognito ลงทะเบียนและยืนยันตัวตนผู้ใช้ใหม่ของแอปพลิเคชัน
- **Amazon API Gateway** (RESTful API) → **AWS Lambda** (การประมวลผลแบบ serverless) → **Amazon DynamoDB** (ฐานข้อมูล NoSQL): ระบบหลังบ้านแบบ serverless ของแอปพลิเคชันรับการเรียก API แบบไดนามิกและประมวลผลตรรกะทางธุรกิจ

หน้า 25 / 65

ตัวอย่าง API Gateway (2 จาก 2)

แผนภาพ Serverless Web Application (Client เชื่อมต่อกับส่วนต่าง ๆ):

- **Amazon Simple Storage Service (S3)** – พื้นที่จัดเก็บแบบออบเจกต์และการโฮสต์เว็บไซต์: Amazon S3 โฮสต์เนื้อหาเว็บไซต์แบบสแตติก เช่น HTML, CSS, JavaScript
- **Amazon Cognito** – ไดรกทอรีผู้ใช้และการยืนยันตัวตน: Amazon Cognito ลงทะเบียนและยืนยันตัวตนผู้ใช้ใหม่ของแอปพลิเคชัน
- **Amazon API Gateway** (RESTful API) → **AWS Lambda** (การประมวลผลแบบ serverless) → **Amazon DynamoDB** (ฐานข้อมูล NoSQL): ระบบหลังบ้านแบบ serverless ของแอปพลิเคชันรับการเรียก API แบบไดนามิกและประมวลผลตรรกะทางธุรกิจ

(สไลด์นี้เน้นส่วนระบบหลังบ้าน API Gateway → Lambda → DynamoDB)

หน้า 26 / 65

โมดูล 5: การประมวลผล (คอนเทนเนอร์และ Serverless)

คอนเทนเนอร์บน AWS (Containers on AWS)

หน้า 27 / 65

แนะนำคอนเทนเนอร์ (Introduction to containers)

คอนเทนเนอร์คือ:

- วิธีการทำเวอร์ชันควบคุมระดับระบบปฏิบัติการ
- แอปพลิเคชันพร้อม dependencies ที่รันได้ในโปรเซสที่แยกทรัพยากรออกจากกัน

(ภาพ: Container 1, Container 2, Container 3)

หน้า 28 / 65

ประโยชน์ของการใช้คอนเทนเนอร์

คอนเทนเนอร์ให้:

- ความสม่ำเสมอของสภาพแวดล้อม (Environment consistency)
- ประสิทธิภาพในการปฏิบัติงาน (Operational efficiency)
- ผลผลิตภาพของนักพัฒนา (Developer productivity)
- การควบคุมเวอร์ชัน (Version control)

หน้า 29 / 65

Docker

Docker คือ:

- แพลตฟอร์มซอฟต์แวร์ที่ช่วยให้คุณสร้าง ทดสอบ และ deploy แอปพลิเคชันได้อย่างรวดเร็ว
- ใช้บรรจุซอฟต์แวร์เป็นหน่วยมาตรฐานที่เรียกว่าคอนเทนเนอร์

คอนเทนเนอร์มีทุกอย่างที่ซอฟต์แวร์ต้องใช้ในการทำงาน: โลบารี (Libraries), เครื่องมือระบบ (System tools), โค้ด (Code), Runtime

หน้า 30 / 65

ตัวอย่างการ deploy ด้วย Docker

ซ้าย: ระบบปฏิบัติการโฮสต์เดียวที่ติดตั้ง Docker และวิธีที่ Docker ทำงานบน on-premises

- ขึ้นจากบนลงล่าง: Application A / B / C → Bins/Libs ของแต่ละแอป → Docker → Host OS → Infrastructure

ขวา: สี่แอปพลิเคชันโฮสต์บนระบบปฏิบัติการ guest สามตัว และวิธีที่ทำงานบน AWS

- Guest OS #1 + Docker: Application A และ B (แต่ละตัวมี Bins/Libs)
- Guest OS #2 + Docker: Application C (มี Bins/Libs)
- Guest OS #3: Application D (มี Bins/Libs, ไม่ใช่ Docker)
- ทั้งหมดอยู่บน Hypervisor → Infrastructure

หน้า 31 / 65

ความท้าทายในการจัดการคอนเทนเนอร์ (1 จาก 2)

ความท้าทายในการจัดการคอนเทนเนอร์ ได้แก่:

- การใช้คอนเทนเนอร์เติบโตอย่างรวดเร็ว
- การหลีกเลี่ยงการกระจายตัวของคอนเทนเนอร์จนควบคุมไม่ได้ (Container sprawl avoidance)
- ความแตกต่างของเวอร์ชัน (Version differences)
- ความเป็นเจ้าของ (Ownership)

หน้า 32 / 65

ความท้าทายในการจัดการคอนเทนเนอร์ (2 จาก 2)

ความท้าทายในการจัดการคอนเทนเนอร์ ได้แก่:

- การจัดวางคอนเทนเนอร์ให้เต็มทรัพยากรอย่างคุ้มค่า (Bin packing)
- คอนเทนเนอร์ที่ไม่จำเป็นแล้ว (zombie containers)
- คอนเทนเนอร์ที่หายไป (Disappearing containers)

หน้า 33 / 65

Amazon Elastic Container Service (Amazon ECS)

Amazon Elastic Container Service (Amazon ECS) คือ:

- บริการจัดการคอนเทนเนอร์ที่ขยายขนาดได้สูงและมีสมรรถนะสูง
- รองรับ Docker containers

(แผนภาพ: Amazon ECS จัดการ Service ซึ่งประกอบด้วย Task ที่มี Container 1, Container 2, Container 3)

หน้า 34 / 65

ประโยชน์ของ Amazon Elastic Container Service (Amazon ECS)

ประโยชน์ของ Amazon ECS ได้แก่:

- บริการจัดการคอนเทนเนอร์ (Container management service)
- การจัดตารางที่ยืดหยุ่น (Flexible scheduling)
- ผสานรวมและขยายต่อได้ (Integrated and extensible)
- ความปลอดภัย (Security)
- สมรรถนะเมื่อขยายขนาด (Performance at scale)
- Task definitions (นิยามของงาน)

หน้า 35 / 65

สาธิต: deploy Docker container บน Amazon Elastic Container Service (Amazon ECS)

หน้า 36 / 65

AWS Fargate

AWS Fargate เป็นเทคโนโลยีสำหรับ Amazon ECS ที่ให้คุณรันคอนเทนเนอร์ได้โดยไม่ต้องจัดการเซิร์ฟเวอร์หรือคลัสเตอร์

- ไม่มีคลัสเตอร์ให้จัดการ (No Clusters to Manage)
- ขยายขนาดได้อย่างราบรื่น (Seamless Scaling)
- ผสานรวมกับ Amazon ECS
- ตัวเลือกการตั้งค่าที่ยืดหยุ่น
- คำนวณตามทรัพยากรที่ใช้ (Resource-based pricing)

AWS Fargate และ ECS stack

ชั้นจากบนลงล่าง:

- ECS Scheduling & Orchestration (การจัดตารางและประสานงาน)
- Placement Engine (กลไกการจัดวาง)
- Cluster Manager (ตัวจัดการคลัสเตอร์)
- Task (หลายตัว) แต่ละตัวทำงานบน ECS Agent, Docker Agent และ ECS AMI

คำอธิบายสี: ส่วน Task จัดการโดยลูกค้า AWS (Managed by AWS Customer); ส่วน ECS Agent, Docker Agent, ECS AMI และชั้นจัดการคลัสเตอร์ จัดการโดย AWS Fargate (Managed by AWS Fargate)

การใช้ AWS Fargate

- สร้าง container image ของคุณ
- ระบุความต้องการ CPU และหน่วยความจำ
- กำหนด IAM และนโยบายเครือข่าย

เมื่อใดควรใช้ AWS Fargate แทน ECS

(สไลด์นี้เน้นฝั่ง Fargate)

ใช้ ECS เมื่อ:

- ใช้ Spot หรือ Reserved Instances
- รัน Windows Clusters
- ต้องการการควบคุมที่ละเอียดขึ้นเพื่อการปฏิบัติตามข้อกำหนด (compliance)
- ต้องการฮาร์ดแวร์เฉพาะทาง

ใช้ Fargate เมื่อ:

- รันคอนเทนเนอร์โดยไม่ต้องจัดการกลุ่ม EC2 instances
- ต้องการเริ่มแอปใหม่อย่างรวดเร็ว
- ย้าย (port) แอปแบบ monolithic
- รับมือกับภาระงานที่ผันผวนสูง

เมื่อใดควรใช้ ECS แทน AWS Fargate

(สไลด์นี้เน้นฝั่ง ECS)

ใช้ ECS เมื่อ:

- ใช้ Spot หรือ Reserved Instances
- รัน Windows Clusters
- ต้องการการควบคุมที่ละเอียดขึ้นเพื่อการปฏิบัติตามข้อกำหนด (compliance)
- ต้องการฮาร์ดแวร์เฉพาะทาง

ใช้ Fargate เมื่อ:

- รันคอนเทนเนอร์โดยไม่ต้องจัดการกลุ่ม EC2 instances
- ต้องการเริ่มแอปใหม่อย่างรวดเร็ว
- ย้าย (port) แอปแบบ monolithic
- รับมือกับภาระงานที่ผันผวนสูง

Kubernetes

Kubernetes เป็นซอฟต์แวร์โอเพนซอร์สสำหรับจัดการและประสานงาน (orchestration) คอนเทนเนอร์

ใช้ Kubernetes เพื่อ:

- จัดการคลัสเตอร์ของ Amazon EC2 compute instances
- รันคอนเทนเนอร์บน instances เหล่านั้น พร้อมกระบวนการสำหรับการ deploy การบำรุงรักษา และการขยายขนาด

ประโยชน์:

- รันแอปพลิเคชันได้ในระดับใหญ่ (Run applications at scale)
- ย้ายแอปพลิเคชันได้อย่างราบรื่น (Seamlessly move applications)
- รันได้ทุกที่ (Run anywhere)
- เพิ่มฟังก์ชันใหม่ (Add new functionality)

Amazon Elastic Container Service for Kubernetes (Amazon EKS)

แผนภาพ (CONTROL PLANE และ NODES) ขั้นตอน:

- จัดเตรียม (provision) Amazon EKS cluster
- deploy worker nodes
- เชื่อมต่อกับ Amazon EKS
- รันแอปพลิเคชันคอนเทนเนอร์

Amazon Elastic Container Registry (Amazon ECR)

Amazon ECR เป็น Docker container registry แบบมีผู้ดูแลให้เต็มรูปแบบ ซึ่งทำให้นักพัฒนาจัดเก็บ จัดการ และ deploy Docker container images ได้ง่าย

- คุณสมบัติ:
- ผสานรวมกับ Amazon elastic container service
 - รองรับ Docker
 - ความพร้อมใช้งานและความทนทานสูง
 - การทำงานร่วมกันเป็นทีม
 - การควบคุมการเข้าถึง (Access control)
 - การเข้ารหัสข้อมูลขณะจัดเก็บ (Encryption at rest)
 - การผสานรวมกับบุคคลที่สาม

(ภาพ: Image → Registry)

ประเด็นสำคัญ (Key Takeaway)

- คอนเทนเนอร์บรรจุทุกอย่างที่ซอฟต์แวร์ต้องใช้ในการทำงาน
- Docker เป็นแพลตฟอร์มซอฟต์แวร์ที่บรรจุซอฟต์แวร์ลงในคอนเทนเนอร์
- แอปพลิเคชันเดียวอาจกระจายอยู่ในหลายคอนเทนเนอร์
- Kubernetes เป็นซอฟต์แวร์โอเพนซอร์สสำหรับประสานงานคอนเทนเนอร์

โมดูล 5: การประมวลผล (คอนเทนเนอร์และ Serverless)

AWS Step Functions

AWS Step Functions

AWS Step Functions ช่วยให้ประสานบริการ AWS เข้าด้วยกันเป็น workflows แบบ serverless

- workflows ประกอบด้วยขั้นตอนต่อเนื่องกัน
- ผลลัพธ์ของขั้นตอนหนึ่งเป็นข้อมูลนำเข้าของขั้นตอนถัดไป
- ตัวอย่าง:
 - AWS Step Functions ช่วยให้ AWS Lambda และ Amazon Elastic Container Service (Amazon ECS) ทำงานร่วมกัน

ประโยชน์:

- สร้างและอัปเดตแอปพลิเคชันแบบกระจาย (distributed applications)
- เขียนโค้ดน้อยลง

AWS Step Functions ทำงานอย่างไร

ขั้นตอนต่อไปนี้จะแสดงการทำงานของ AWS Step Functions:

- กำหนดขั้นตอนใน workflow
- ตั้งค่า task เพื่อประสานการทำงานของบริการ AWS
- รันได้ตามต้องการนานสูงสุด 1 ปี (maximum runtime timeout)
- สลับเปลี่ยน tasks หรือขั้นตอนได้

สรุป

- วัตถุประสงค์และการทำงานของ AWS Lambda
- วัตถุประสงค์และการทำงานของ application programming interfaces (APIs) รวมถึง RESTful APIs
- ประโยชน์และการทำงานของ Amazon API Gateway
- วัตถุประสงค์และการทำงานของคอนเทนเนอร์และบริการ AWS ที่รองรับการใช้คอนเทนเนอร์

- วัดประสิทธิภาพและการทำงานของ AWS Step Functions เพื่อจบโมดูลนี้ ให้ทำ:
- ตรวจสอบความรู้ (Knowledge check)

หน้า 49 / 65

กลยุทธ์การทำข้อสอบ (Test Strategies)

หน้า 50 / 65

สถานการณ์ของโมดูล 5 (Module 5 Scenario)

บริษัทขนส่งด้วยรถบรรทุกแห่งหนึ่งต้องการลดการพึ่งพาผู้ให้บริการคลาวด์ปัจจุบัน เพื่อไม่ให้ถูก "ผูกขาด" (locked in) พร้อมกับเพิ่มความเร็วในการพัฒนาแอปพลิเคชันต่าง ๆ บริษัทกำลังตัดสินใจว่าควรใช้คอนเทนเนอร์หรือใช้ serverless พวกเขาต้องการความจุในการประมวลผลที่ปลอดภัยเฉพาะตอนที่รถบรรทุกอยู่ที่ฐานหลัก ซึ่งอาจกินเวลาหนึ่งวัน เดือนละสองสามครั้ง บางครั้งพวกเขาสนับสนุนบริษัทพันธมิตรเมื่อมาที่เมือง จึงอาจต้องรองรับผู้ใช้เหล่านั้นทุก ๆ สองสามเดือน

ข้อความใดอธิบายสถานการณ์ที่บริษัทอาจพบว่าคอนเทนเนอร์เป็นตัวเลือกที่เหมาะสมกว่า? เลือกสามคำตอบ

- A. สามารถลดขนาดหรือปรับขนาดได้ตามต้องการ
- B. ต้องการการจัดการทรัพยากรและนโยบายที่เกี่ยวข้อง รวมถึงกำกับดูแลความปลอดภัยเอง
- C. ต้องการให้ผู้ให้บริการคลาวด์จัดเตรียมโครงสร้างพื้นฐานและจัดการความจุให้
- D. ย้ายซอฟต์แวร์จากสภาพแวดล้อมหนึ่งไปอีกสภาพแวดล้อมหนึ่งได้ด้วยการแยกส่วน (isolation)
- E. มีความต้องการด้านการพกพา (portability) และคงความเป็นกลางต่อผู้ให้บริการ (vendor-agnosticism)

หน้า 51 / 65

สถานการณ์ของโมดูล 5 – คำสำคัญ

บริษัทขนส่งด้วยรถบรรทุกแห่งหนึ่งต้องการลดการพึ่งพา (minimize dependence) ผู้ให้บริการคลาวด์ปัจจุบัน เพื่อไม่ให้ถูก "ผูกขาด" (not "locked in") พร้อมกับเพิ่มความเร็วในการพัฒนา (increasing their development velocity) แอปพลิเคชันต่าง ๆ บริษัทกำลังพยายาม (is trying) ตัดสินใจว่าควรใช้คอนเทนเนอร์ (containers) หรือใช้ serverless พวกเขาต้องการความจุในการประมวลผลที่ปลอดภัยเฉพาะ (only need secure) ตอนที่รถบรรทุกอยู่ที่ฐานหลัก ซึ่งอาจกินเวลาหนึ่งวันเดือนละสองสามครั้ง (a couple times) บางครั้งพวกเขาสนับสนุนบริษัทพันธมิตร (support partner companies) เมื่อมาที่เมือง จึง (town; therefore) อาจต้องรองรับผู้ใช้เหล่านั้นทุก ๆ สองสามเดือน

ข้อความใดอธิบาย (describe) สถานการณ์ที่บริษัทอาจพบว่าคอนเทนเนอร์ (find that containers) เป็นตัวเลือกที่เหมาะสมกว่า (more ideal option)? เลือกสามคำตอบ

(คำที่เป็นตัวหนาคือคำสำคัญที่ไฮไลต์ไว้ในสไลด์ ตัวเลือกถูกปิดไว้)

- A. สามารถลดขนาดหรือปรับขนาดได้ตามต้องการ
- B. ต้องการการจัดการทรัพยากรและนโยบายที่เกี่ยวข้อง รวมถึงกำกับดูแลความปลอดภัยเอง
- C. ต้องการให้ผู้ให้บริการคลาวด์จัดเตรียมโครงสร้างพื้นฐานและจัดการความจุให้
- D. ย้ายซอฟต์แวร์จากสภาพแวดล้อมหนึ่งไปอีกสภาพแวดล้อมหนึ่งได้ด้วยการแยกส่วน (isolation)
- E. มีความต้องการด้านการพกพา (portability) และคงความเป็นกลางต่อผู้ให้บริการ (vendor-agnosticism)

หน้า 52 / 65

สถานการณ์ของโมดูล 5 – คำตอบ

บริษัทขนส่งด้วยรถบรรทุกแห่งหนึ่งต้องการลดการพึ่งพาผู้ให้บริการคลาวด์ปัจจุบัน เพื่อไม่ให้ถูก "ผูกขาด" พร้อมกับเพิ่มความเร็วในการพัฒนาแอปพลิเคชันต่าง ๆ บริษัทกำลังตัดสินใจว่าควรใช้คอนเทนเนอร์หรือใช้ serverless พวกเขาต้องการความจุในการประมวลผลที่ปลอดภัยเฉพาะตอนที่รถบรรทุกอยู่ที่ฐานหลัก ซึ่งอาจกินเวลาหนึ่งวัน เดือนละสองสามครั้ง บางครั้งพวกเขาสนับสนุนบริษัทพันธมิตรเมื่อมาที่เมือง จึงอาจต้องรองรับผู้ใช้เหล่านั้นทุก ๆ สองสามเดือน

ข้อความใดอธิบายสถานการณ์ที่บริษัทอาจพบว่าคอนเทนเนอร์เป็นตัวเลือกที่เหมาะสมกว่า? เลือกสามคำตอบ

- A. สามารถลดขนาดหรือปรับขนาดได้ตามต้องการ

- B. ต้องการการจัดการทรัพยากรและนโยบายที่เกี่ยวข้อง รวมถึงกำกับดูแลความปลอดภัยเอง (ไฮไลต์ในสไลด์)
- C. ต้องการให้ผู้ให้บริการคลาวด์จัดเตรียมโครงสร้างพื้นฐานและจัดการความจุให้
- D. ย้ายซอฟต์แวร์จากสภาพแวดล้อมหนึ่งไปอีกสภาพแวดล้อมหนึ่งได้ด้วยการแยกส่วน (isolation)
- E. มีความต้องการด้านการพกพาและคงความเป็นกลางต่อผู้ให้บริการ (ไฮไลต์ในสไลด์)

หมายเหตุผู้แปล: ในสไลด์ต้นฉบับมีแถบไฮไลต์สามแถบ คือที่ข้อ B, ข้อ E และ แถบว่างหนึ่งแถบใต้ข้อ E ซึ่งไม่ตรงกับตัวเลือกใด (แถบไฮไลต์ในสไลด์นี้ดูเลื่อนตำแหน่ง) จึงระบุได้ชัดเจนเพียง B และ E ตามที่ปรากฏ ควรตรวจสอบคำตอบข้อที่สามกับผู้อ่าน

หน้า 53 / 65

สถานการณ์โปรเจกต์การปฏิบัติการระบบ (Systems Operations Project Scenario)

Mom & Pop Café & Bakery

หน้า 54 / 65

AWS กับโลกจริง

Amazon Web Services (AWS) มีบริการหลากหลายหมวดหมู่ เช่น:

- Compute (การประมวลผล)
- Storage (การจัดเก็บข้อมูล)
- Database (ฐานข้อมูล)
- Analytics (การวิเคราะห์ข้อมูล)

AWS มอบโซลูชันบนคลาวด์ให้ทั้งองค์กรขนาดใหญ่และธุรกิจขนาดเล็ก เพื่อขับเคลื่อนการดำเนินงานไปข้างหน้า:

- ได้เร็วขึ้น
- ด้วยต้นทุนที่ลดลง
- พร้อมความสามารถในการขยายขนาดตามความต้องการ

หน้า 55 / 65

วัตถุประสงค์ของโปรเจกต์

เมื่อทำโปรเจกต์นี้เสร็จ คุณจะสามารรถ:

- สัมผัสความท้าทายด้านการสื่อสารที่เกิดขึ้นเมื่อพยายามใช้เทคโนโลยีเป็นทางแก้ปัญหาทางธุรกิจ
- แปลงความต้องการของลูกค้าให้เป็นโซลูชันทางเทคนิคที่เสนอ
- นำเสนอโซลูชันที่เสนอต่อลูกค้า
- ใช้ AWS Best Practices เพื่อให้ได้โซลูชันที่ประหยัด ทนทาน และปลอดภัย

หน้า 56 / 65

Mom & Pop Café & Bakery: การเตรียมตัวพบลูกค้า (Customer meeting preparation)

หน้า 57 / 65

Mom & Pop Café & Bakery

คำขอให้ส่งประวัติคำสั่งซื้อรายวัน (Request to send daily order history)

หน้า 58 / 65

ความต้องการทางธุรกิจ: การรายงานอัตโนมัติ

- Sophie ต้องการส่งรายงานประจำวันให้แม่และพ่อ รายงานนี้จะแสดงรายการสิ่งที่ร้านต้องอบในแต่ละวัน
- รายงานจะใช้คำสั่งซื้อก่อนหน้าเพื่อกำหนดความต้องการในการอบของวันพรุ่งนี้

หน้า 59 / 65

กิจกรรมที่ 5: สร้าง AWS Lambda function เพื่อส่งรายงานประจำวันทางอีเมล

หน้า 60 / 65

กิจกรรมที่ 5: แผนภาพสถาปัตยกรรมและลำดับการทำงานของรายงานวิเคราะห์ยอดขาย

แผนภาพ (AWS Cloud → Region):

- 1. salesAnalysisReportDailyTrigger (Amazon CloudWatch event – ทริกเกอร์ทุกวันเวลา 20:00 วันจันทร์-เสาร์) → เรียก salesAnalysisReport

Lambda function (ใช้ salesAnalysisReportRole IAM role)

- 2. salesAnalysisReport เรียก salesAnalysisReportDataExtractor Lambda function (ใช้ PyMySQL library ใน Lambda layer และใช้ salesAnalysisReportDERole IAM role)
- 3. salesAnalysisReportDataExtractor ดึงข้อมูลจากฐานข้อมูล mom_pop_db (MariaDB – MySQL) บน mompopcafe EC2 LAMP instance (อยู่ใน VPC → Subnet → Security group; EC2 instance contents)
- 4. ส่งข้อมูลกลับไปยัง salesAnalysisReport
- 5. salesAnalysisReport ส่งรายงานไปยัง salesAnalysisReportTopic (Amazon Simple Notification Service topic)
- 6. SNS ส่งอีเมลไปยังที่อยู่อีเมลของพ่อ (Pop's email address)

หมายเหตุผู้แปล: ข้อความที่ดึงจาก PDF ต้นฉบับหน้านี้มีข้อความภาษาญี่ปุ่น "この画像は表示できません。" (แปลว่า "ไม่สามารถแสดงภาพนี้ได้") ปนอยู่ ซึ่งเป็นข้อความ placeholder ของไอคอนที่แสดงไม่ได้ ไม่ใช่เนื้อหาบทเรียน

หน้า 61 / 65

เริ่มกิจกรรมที่ 5: สร้าง Lambda function เพื่อส่งรายงานทางอีเมล

หน้า 62 / 65

AWS Lambda

AWS Lambda อาจแก้ความต้องการทางธุรกิจอื่นใดให้ Mom & Pop Café ได้อีกบ้าง?

หน้า 63 / 65

AWS Lambda

AWS Lambda อาจแก้ความต้องการทางธุรกิจอื่นใดให้ Mom & Pop Café ได้อีกบ้าง?

แนวทางแก้ (Solution): AWS Lambda function สามารถดึงข้อมูลการเข้าถึงเว็บจากไฟล์ log แล้วส่งข้อมูลที่เกี่ยวข้องให้แม่และพ่อ

หน้า 64 / 65

เวลาทำโปรเจกต์: ฐานความรู้สำหรับการแก้ไขปัญหา

~ 30 นาที

หน้า 65 / 65

ขอบคุณ (Thank you)

ข้อความลิขสิทธิ์ท้ายสไลด์: ห้ามทำซ้ำหรือเผยแพร่ซ้ำโดยไม่ได้รับอนุญาตเป็นลายลักษณ์อักษรจาก Amazon Web Services, Inc. ห้ามคัดลอก ให้อืม หรือขายเพื่อการค้า หากมีข้อแก้ไขหรือข้อเสนอแนะเกี่ยวกับคอร์ส โปรดอีเมลมาที่ aws-course-feedback@amazon.com สำหรับคำถามอื่น ๆ ทั้งหมด ติดต่อที่ <https://aws.amazon.com/contact-us/aws-training/> เครื่องหมายการค้าทั้งหมดเป็นทรัพย์สินของเจ้าของ